



MIDDLE EAST TECHNICAL UNIVERSITY

MMI714

Generative Models for Multimedia

Introduction to Generative
Adversarial Networks - GANs

Welcome

This is MMI714

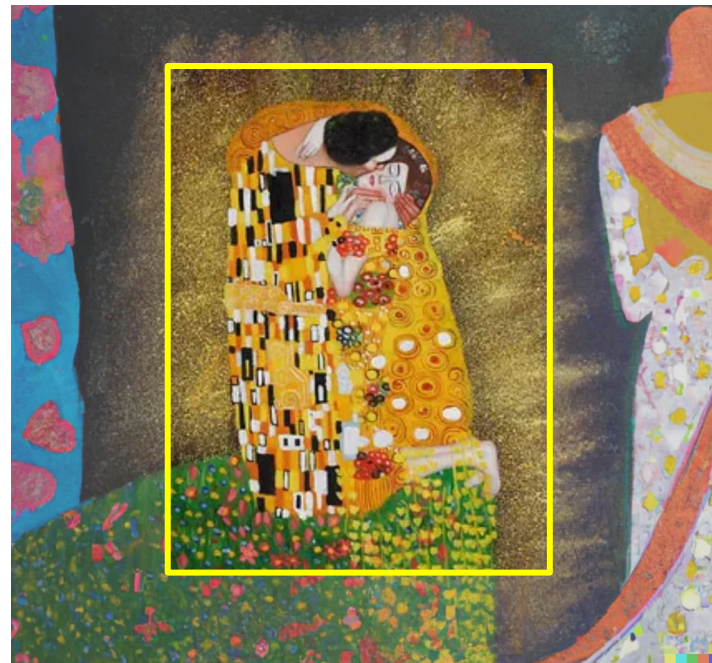
“Generative Models for Multimedia”

This week we will talk about:

Adversarial Training,

Vanilla GAN, min-max optimization,

DCGAN, Conditioning GANs



Unsupervised Training

- Generative modeling (as we discussed up to this point) is an unsupervised learning task in machine learning that involves automatically discovering and learning data distributions.
- Or more specifically: learning *"the regularities or patterns in input data in such a way that the model can be used to generate or output new examples that plausibly could have been drawn from the original dataset."*

Unsupervised Training

- Generative modeling (as we discussed up to this point) is an unsupervised learning task in machine learning that involves automatically discovering and learning data distributions.
- Or more specifically: learning *"the regularities or patterns in input data in such a way that the model can be used to generate or output new examples that plausibly could have been drawn from the original dataset."*
- In other words:

We used only positive examples and no labels up to now.

Supervised Generation?

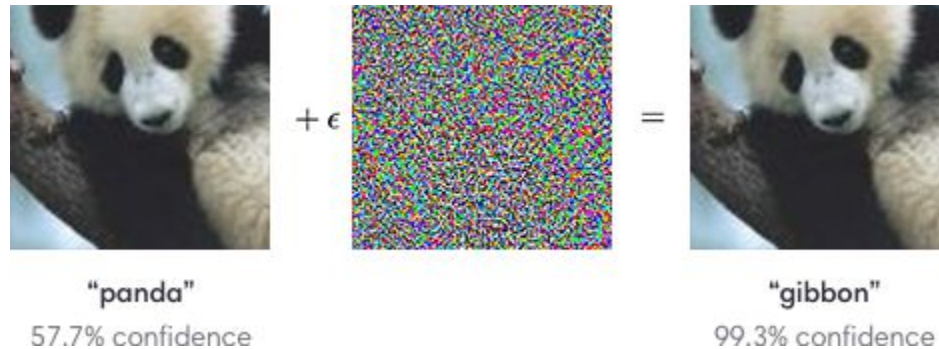
- GANs are a clever way of training a generative model by framing the problem as a supervised learning problem
- with two sub-models:
 - the generator model that we train to generate new examples, and
 - the discriminator model that tries to classify examples as either real (from the domain) or fake (generated).
- The two models are trained together in a zero-sum game, (i.e. **adversarial**), until the discriminator model is fooled about half the time, meaning the generator model is generating plausible examples.

Adversarial Learning?

- (is another issue)
- Adversarial machine learning [Wiki] is the study of the attacks on machine learning algorithms, and of the defenses against such attacks.
- Most machine learning techniques are mostly designed to work on specific problem sets, under the assumption that the training and test data are generated from the same statistical distribution.
- However, this assumption is often dangerously violated in practical high-stake applications, where users may intentionally supply fabricated data that violates the statistical assumption.

Adversarial Learning?

- Adversarial examples are malicious inputs purposely designed to fool a machine learning model.



- So, we can use the adversarial samples to further train our classifier (i.e. adversarial augmentation).

figure from [Explaining and Harnessing Adversarial Examples](#),
Goodfellow et al, ICLR 2015.

Adversarial **Generation**?

- How to use the adversarial dynamics for generation?
- We need two additional skills:
 - We need adversarial samples (supervised in GANs)
 - We need to be able to tell if the sample is adversarial (classification)

Adversarial **Generation**?

- Let's operationalize the idea:
- It's like having an artist and an art critic.
- The artist tries to make paintings that look real, and the critic tries to spot which ones are fake.
- They keep challenging each other, and over time, the artist becomes so good that the critic can't always tell the real paintings from the fake ones.
- It's a friendly competition that makes the artist really skilled at creating realistic art!



Adversarial Generation?

- The adversarial learning here is the friendly competition between the "creator" (generator) and the "judge" (discriminator or classifier).
- The creator gets better at making realistic things, and the judge gets better at telling what's real or fake.
- They push each other to improve, and that's the adversarial part—the constant challenge and competition that leads to better and more realistic creations.



Adversarial Generation?

- GANs are a utilization of this idea.
- There are two models:
 - the generator model that we train to generate new examples,
 - and the discriminator model that tries to classify examples as either real (from the domain) or fake (generated).
- The two models are trained together in a zero-sum game, adversarial, until the discriminator model is fooled about half the time, meaning the generator model is generating plausible examples.



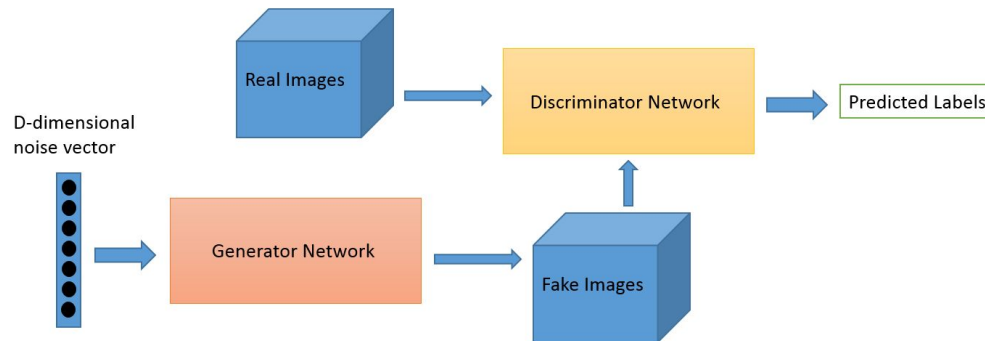
Adversarial Generation?

- OK so GANs are an architecture for automatically training a generative model by treating the unsupervised problem as supervised and using both a generative and a discriminative model.
- We have so many questions to answer:
 - What do the generator/discriminator look like?
 - Is there a latent space?
 - What is the loss function for it?



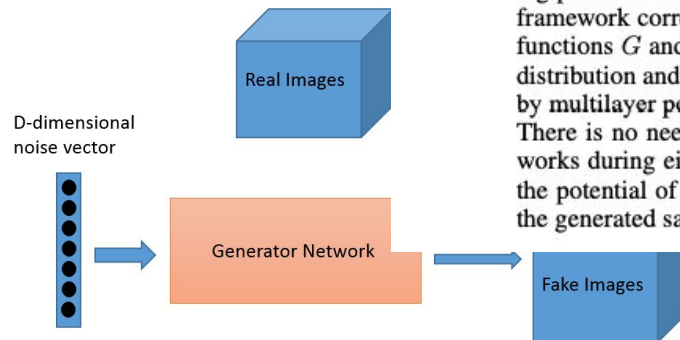
Generative Adversarial Nets

- The GAN architecture was first described in the 2014 paper by Ian Goodfellow, et al. titled “Generative Adversarial Networks.”
- Goodfellow’s original GAN architecture used fully connected layers in both the generator and discriminator.
- The generator took random noise as input.



Generative Adversarial

- The GAN architecture was first introduced by Goodfellow, et al. titled "Generative Adversarial Networks"
- Goodfellow's original GAN architecture involves both the generator and discriminator
- The generator takes random noise as input



Generative Adversarial Nets

Ian J. Goodfellow, Jean Pouget-Abadie*, Mehdi Mirza, Bing Xu, David Warde-Farley,
 Sherjil Ozair†, Aaron Courville, Yoshua Bengio†
 Département d'informatique et de recherche opérationnelle
 Université de Montréal
 Montréal, QC H3C 3J7

Abstract

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G . The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions G and D , a unique solution exists, with G recovering the training data distribution and D equal to $\frac{1}{2}$ everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.

Generative Adversarial Nets

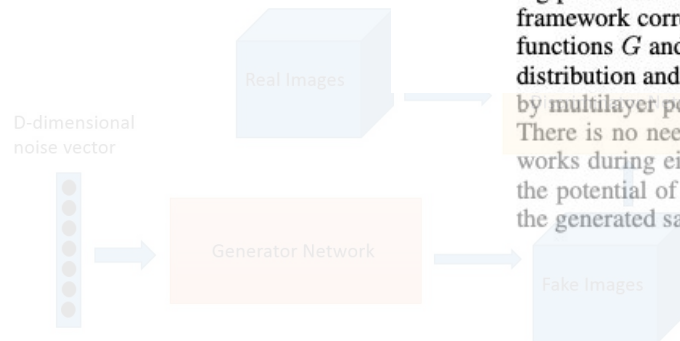
Ian J. Goodfellow, Jean Pouget-Abadie*, Mehdi Mirza, Bing Xu, David Warde-Farley,
Sherjil Ozair†, Aaron Courville, Yoshua Bengio†

Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 1J7

- The GAN architecture was first described in the 2014 paper by Ian Goodfellow, et al. titled “Generative Adversarial Networks.”
- Goodfellow’s original GAN architecture used fully connected layers in both the generator and discriminator.
- The generator took random

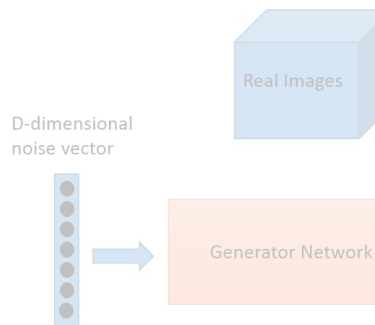
Abstract

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G . The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions G and D , a unique solution exists, with G recovering the training data distribution and D equal to $\frac{1}{2}$ everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.



Generative Adversarial

- The GAN architecture was first proposed by Goodfellow, et al. titled "Generative Adversarial Networks"
- Goodfellow's original GAN architecture consists of both the generator and discriminator
- The generator takes random noise as input



Abstract

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G . The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions G and D , a unique solution exists, with G recovering the training data distribution and D equal to $\frac{1}{2}$ everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.

1 Introduction

The promise of deep learning is to discover rich, hierarchical models [2] that represent probability distributions over the kinds of data encountered in artificial intelligence applications, such as natural images, audio waveforms containing speech, and symbols in natural language corpora. So far, the most striking successes in deep learning have involved discriminative models, usually those that map a high-dimensional, rich sensory input to a class label [14, 22]. These striking successes have primarily been based on the backpropagation and dropout algorithms, using piecewise linear units [19, 9, 10] which have a particularly well-behaved gradient. Deep *generative* models have had less of an impact, due to the difficulty of approximating many intractable probabilistic computations that arise in maximum likelihood estimation and related strategies, and due to difficulty of leveraging the benefits of piecewise linear units in the generative context. We propose a new generative model estimation procedure that sidesteps these difficulties. ¹

Generative Adversarial Nets

- Our job is the same as always:

“to learn the generator’s distribution p_g over some data x ”

3 Adversarial nets

The adversarial modeling framework is most straightforward to apply when the models are both multilayer perceptrons. To learn the generator’s distribution p_g over data x , we define a prior on input noise variables $p_z(z)$, then represent a mapping to data space as $G(z; \theta_g)$, where G is a differentiable function represented by a multilayer perceptron with parameters θ_g . We also define a second multilayer perceptron $D(x; \theta_d)$ that outputs a single scalar. $D(x)$ represents the probability that x came from the data rather than p_g . We train D to maximize the probability of assigning the correct label to both training examples and samples from G . We simultaneously train G to minimize $\log(1 - D(G(z)))$:

In other words, D and G play the following two-player minimax game with value function $V(G, D)$:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]. \quad (1)$$

Generative Adversarial Nets

This is some kind of BCE Loss

$$-\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

- Our job is the same as always:

“to learn the generator’s distribution p_g over some data x ”

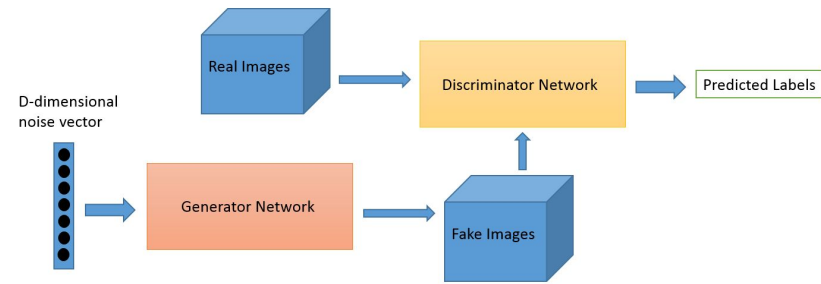
3 Adversarial nets

The adversarial modeling framework is most straightforward to apply when the models are both multilayer perceptrons. To learn the generator’s distribution p_g over data x , we define a prior on input noise variables $p_z(z)$, then represent a mapping to data space as $G(z; \theta_g)$, where G is a differentiable function represented by a multilayer perceptron with parameters θ_g . We also define a second multilayer perceptron $D(x; \theta_d)$ that outputs a single scalar. $D(x)$ represents the probability that x came from the data rather than p_g . We train D to maximize the probability of assigning the correct label to both training examples and samples from G . We simultaneously train G to minimize $\log(1 - D(G(z)))$:

In other words, D and G play the following two-player minimax game with value function $V(G, D)$:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]. \quad (1)$$

Generative Adversarial Nets



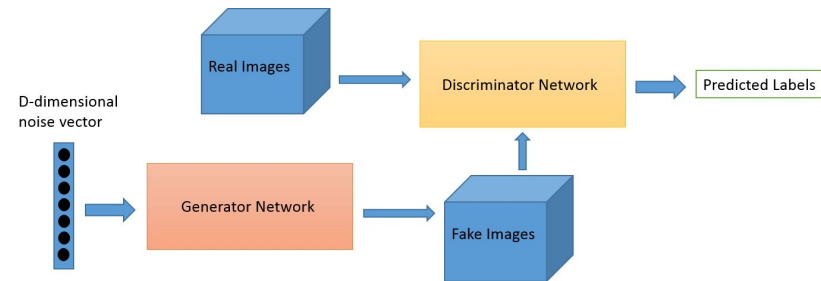
- Our job is the same as always:

“to learn the generator’s distribution p_g over some data x ”

$$\min_G \max_D V(D, G) = \underbrace{\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})]}_{\text{Component 1}} + \underbrace{\mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]}_{\text{Component 2}}.$$

- Component 1: updates *only the D’s weights to better distinguish real data*.
- Component 2: updates both G and D’s *weights are updated*.

Generative Adversarial Nets



- Our job is the same as always:

“to learn the generator’s distribution p_g over some data x ”

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by **ascending** its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by **descending** its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

min-max optimization?

- How are the weights are updated in each iteration?
 - First the discriminator:

$$\theta_{\text{disc_new}} = \theta_{\text{disc_old}} \oplus \eta \cdot \nabla_{\theta_{\text{disc}}} V(D, G)$$


$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

min-max optimization?

- How are the weights are updated in each iteration?
 - First the discriminator:

$$\theta_{\text{disc_new}} = \theta_{\text{disc_old}} \oplus \eta \cdot \nabla_{\theta_{\text{disc}}} V(D, G)$$

- Then the generator

$$\theta_{\text{gen_new}} = \theta_{\text{gen_old}} \ominus \eta \cdot \nabla_{\theta_{\text{gen}}} V(D, G)$$

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

min-max optimization?

- In the rest of the paper they show that the convergence criteria is the discriminator model being fooled about half the time:

4.1 Global Optimality of $p_g = p_{\text{data}}$

We first consider the optimal discriminator D for any given generator G .

Proposition 1. For G fixed, the optimal discriminator D is

$$D_G^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})} = 1/2 \quad (2)$$

Proof. The training criterion for the discriminator D , given any generator G , is to maximize the quantity $V(G, D)$

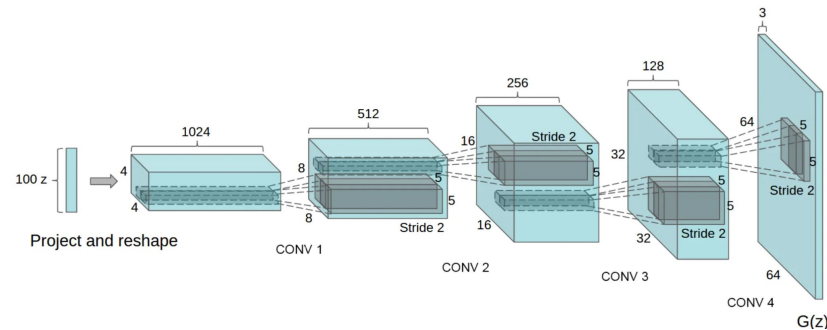
...

DCGAN

- In the past decade of deep learning, whenever there is a new idea/model with fully connected layers, the first effort has always been to make the architecture fully convolutional (by, unsurprisingly, computer vision scientist)
 - Semantic Segmentation → FCN, UNet
 - Detection → YOLO v1
 - GANs → DCGAN

DCGAN

- DCGAN is a GAN with fully-convolutional generator. The generator mainly composes of convolution layers
 - without max pooling or fully connected layers.
- DCGAN uses convolutional stride and transposed convolution for the downsampling and the upsampling.





DCGAN

- More on fundamental points (from [paperswithcode](#))
 - Replacing any pooling layers with strided convolutions (discriminator) and fractional-strided convolutions (generator).
 - Using batchnorm in both the generator and the discriminator.
 - Removing fully connected hidden layers for deeper architectures.
 - Using ReLU activation in generator for all layers except for the output, which uses tanh.
 - Using LeakyReLU activation in the discriminator for all layer.

Alec Radford & Luke Metz
Indico Research
Boston, MA
{alec, luke}@indico.io

Soumith Chintala
Facebook AI Research
New York, NY
soumith@fb.com

ABSTRACT

In recent years, supervised learning with convolutional networks (CNNs) has seen huge adoption in computer vision applications. Comparatively, unsupervised learning with CNNs has received less attention. In this work we hope to help bridge the gap between the success of CNNs for supervised learning and unsupervised learning. We introduce a class of CNNs called deep convolutional generative adversarial networks (DCGANs), that have certain architectural constraints, and demonstrate that they are a strong candidate for unsupervised learning. Training on various image datasets, we show convincing evidence that our deep convolutional adversarial pair learns a hierarchy of representations from object parts to scenes in both the generator and discriminator. Additionally, we use the learned features for novel tasks - demonstrating their applicability as general image representations.



Significance?

- GANs for vision: the paper introduces architectural changes that greatly improve the stability and effectiveness of training GANs for image generation tasks.
- For a while any vision-based GAN architecture continued to be a successor of DCGANs (until transformers)
- Directly applicable to other signals (sound etc.)

Alec Radford & Luke Metz
indico Research
Boston, MA
{alec, luke}@indico.io

Soumith Chintala
Facebook AI Research
New York, NY
soumith@fb.com

ABSTRACT

In recent years, supervised learning with convolutional networks (CNNs) has seen huge adoption in computer vision applications. Comparatively, unsupervised learning with CNNs has received less attention. In this work we hope to help bridge the gap between the success of CNNs for supervised learning and unsupervised learning. We introduce a class of CNNs called deep convolutional generative adversarial networks (DCGANs), that have certain architectural constraints, and demonstrate that they are a strong candidate for unsupervised learning. Training on various image datasets, we show convincing evidence that our deep convolutional adversarial pair learns a hierarchy of representations from object parts to scenes in both the generator and discriminator. Additionally, we use the learned features for novel tasks - demonstrating their applicability as general image representations.



Significance?

- GANs for vision: the paper introduces architectural changes that greatly improve the stability and effectiveness of training GANs for image generation tasks.
- For a while any vision-based GAN architecture continued to be a successor of DCGANs (transformers)
- Directly applicable to other signals (source)

Alec Radford & Luke Metz
Indico Research
Boston, MA
{alec, luke}@indico.io

Soumith Chintala
Facebook AI Research
New York, NY
soumith@fb.com

ABSTRACT

In recent years, supervised learning with convolutional networks (CNNs) has seen huge adoption in computer vision applications. Comparatively, unsupervised learning with CNNs has received less attention. In this work we hope to help bridge the gap between the success of CNNs for supervised learning and unsupervised learning. We introduce a class of CNNs called deep convolutional generative

Workshop track - ICLR 2018

SYNTHESIZING AUDIO WITH GANs

Chris Donahue*, Julian McAuley[†] & Miller Puckette*
Departments of Music* and Computer Science[†]
University of California, San Diego
{cdonahue, jmcauley, msp}@ucsd.edu

ABSTRACT

While Generative Adversarial Networks (GANs) have seen wide success at the problem of synthesizing realistic images, they have seen little application to audio generation. In this paper, we introduce WaveGAN, a first attempt at applying GANs to raw audio synthesis in an unsupervised setting. Our experiments on speech demonstrate that WaveGAN can produce intelligible words from a small vocabulary of human speech, as well as synthesize audio from other domains such as bird vocalizations, drums, and piano. Qualitatively, we find that human judges prefer the generated examples from WaveGAN over those from a method which naïvely applies GANs on image-like audio feature representations.

Conditioning (first general)

- In the context of generative models, conditioning typically involves **providing some information or context** to guide the generation process.
- For example, in image generation, you could condition the model on specific features or attributes you want to be present in the generated image.
- Conditioning is often represented mathematically using the vertical bar notation (like conditional probability).
 - For instance, in a probability distribution $P(X|Y)$, it denotes the probability of X given the condition Y .

Unsupervised Clustering of Seismic Signals Using Deep Convolutional Autoencoders

S. Mostafa Mousavi[✉], Weiqiang Zhu[✉], William Ellsworth[✉], and Gregory Beroza

Conditioning (first general)

- **All** probabilistic generative models, like Gaussian Mixture Models (GMMs) or Variational Autoencoders (VAEs), can be conditioned on input variables to generate samples that satisfy certain criteria.

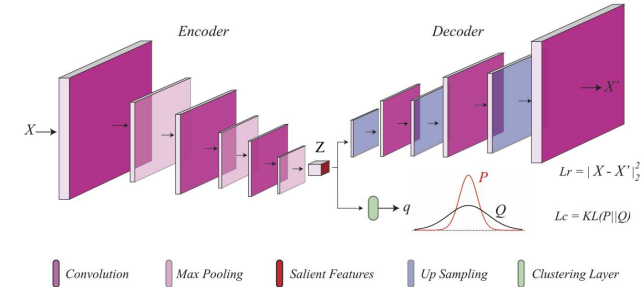
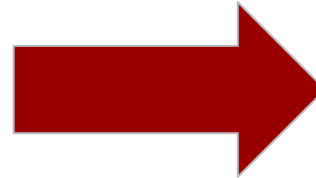


Fig. 1. Network architecture. The encoder and decoder are composed of fully convolutional layers followed by max-pooling and up-sampling layers, respectively. Reconstruction loss (L_r) and the clustering loss (L_c) are given in the figure.

$$L = (1 - \lambda)L_r + \lambda L_c \quad (1)$$

$$q_{ij} = \frac{(1 + \|z_i - \mu_j\|^2)^{-1}}{\sum_j (1 + \|z_i - \mu_j\|^2)^{-1}} \quad (2)$$

The membership probabilities are used to compute an auxiliary target distribution, p_{ij}

$$p_{ij} = \frac{q_{ij}^2 / \sum_i q_{ij}}{\sum_j (q_{ij}^2 / \sum_i q_{ij})} \quad (3)$$

and clustering is performed by minimizing the Kullback-Leibler (KL) divergence between the soft assignments, q_{ij} , and the target distribution, p_{ij}

$$L_c = KL(P \parallel Q) = \sum_i \sum_j p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right) \quad (4)$$

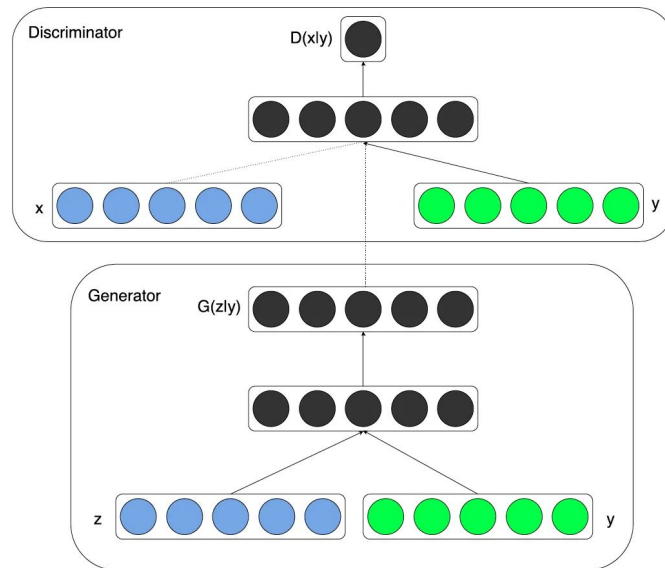


How to condition? (first general)

1. **Input to the Model:** This involves providing additional information directly to the model (which is mostly the decoder, but also the discriminator for GANs).
2. **Shaping the Latent Space:** Conditioning can also be achieved by shaping the latent space. This means that instead of directly modifying the input, you manipulate the latent representation of the data.
3. **Shaping the Loss Function:** Modifying the loss function is indeed a valid way to condition a model. This can involve adding extra terms to the loss function to prioritize certain aspects of the generated output.
4. **Shaping the Input:** Manipulating the input may also condition a model.
5. **Architecture Specific Methods...**

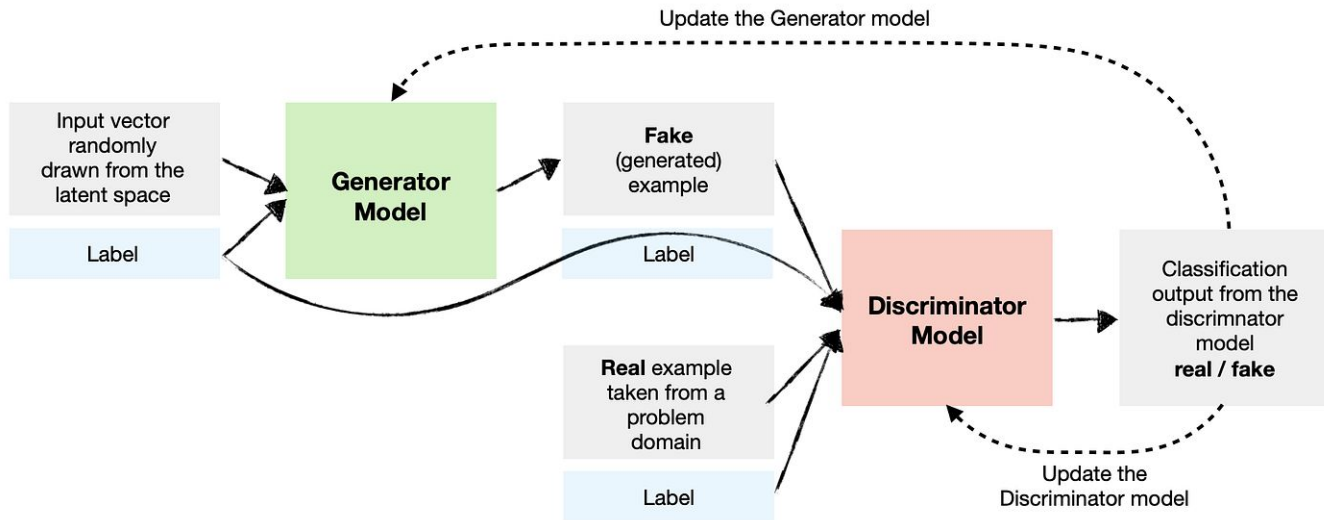
Conditional GANs $\rightarrow$ cGANs

- In conditional GANs the usual practise is some additional information being passed to the generator and/or discriminator networks.
- In the simplest case, the information usually represents classes in the training set,
 - for example, different digits in the case of MNIST data set.



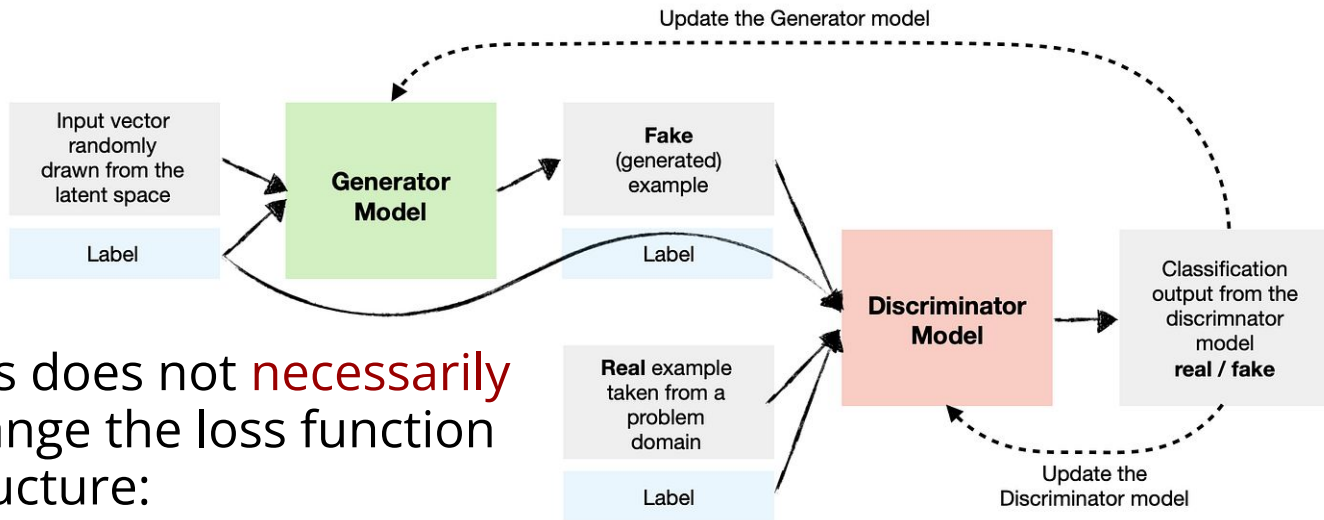
Conditional GANs $\rightarrow$ cGANs

- In conditional GANs the usual practise is some additional information being passed to the generator and/or discriminator networks.



Conditional GANs $\rightarrow$ cGANs

- In conditional GANs the usual practise is some additional information being passed to the generator and/or discriminator networks.



- This does not **necessarily** change the loss function structure:

$$\mathcal{L}_{cGAN} = \mathbb{E}_{x,y}[\log D(x,y)] + \mathbb{E}_z[\log(1 - D(G(z,y), y))]$$

Conditional GANs $\rightarrow$ cGANs

- In conditional GANs the usual practise is some additional information being passed to the generator and/or discriminator networks.

But it may!

- This does not necessarily change the loss function structure:
- In cGANs, during training, you may provide both correct and "wrong" labels to the discriminator as part of the adversarial training process.
- The objective is to train the discriminator to distinguish between real samples with their correct conditions and generated samples with incorrect conditions.

$$\mathcal{L}_{cGAN} = \mathbb{E}_{x,y}[\log D(x,y)] + \mathbb{E}_z[\log(1 - D(G(z,y), y))]$$

Conditional GANs $\rightarrow$ cGANs

- In conditional GANs the usual practise is some additional information being passed to the generator and/or discriminator networks.

- In that case, for false examples fed to the Discriminator, the loss component becomes: $\log(1-D(x))$

But it may!

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

- This does not necessarily change the loss function structure:

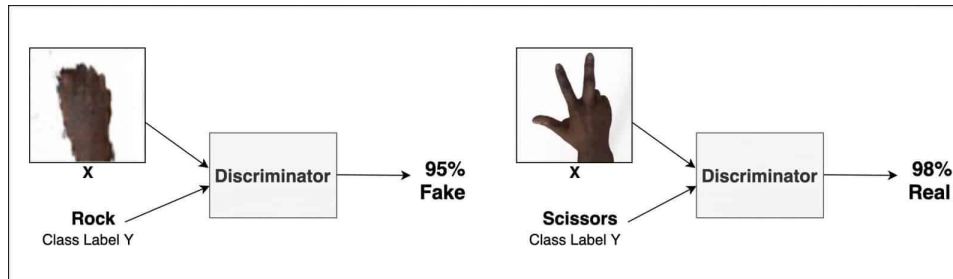
$$\mathcal{L}_{cGAN} = \mathbb{E}_{x,y} [\log D(x, y)] + \mathbb{E}_z [\log(1 - D(G(z, y), y))]$$

cGANs Intuition

- The Generator uses the noise vector z and the label y to synthesize a fake example $G(z, y) = x|y$
(x conditioned on y , where x is the generated fake example).
- This fake example aims to fool the discriminator by looking as similar as possible to a real example for the given label.
- *It is now not enough anymore for the Generator to produce realistic-looking data; it is equally important that the generated examples also match the label.*

cGANs Intution

- The Discriminator is fed both real and fake examples with labels. It learns to not just recognize real data from fake, but also zeroes onto matching pairs.
- A pair is matching when the image has a correct label assigned to it. The Discriminator finally outputs a probability indicating the input is real or fake. Its goal is to learn to:



GAN Loss function problems

- The standard GAN loss function, also known as the min-max loss, was this:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))].$$

- The generator tries to minimize this function while the discriminator tries to maximize it. Looking at it as a min-max game.
- In practice, this does not work perfect.
- This loss saturates for the Generator: *meaning that the generator quite frequently stops training if it doesn't catch up with the discriminator.*

GAN Loss function problems

- While the generator is trained, it samples random noise and produces an output from that noise.

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

The output then goes through the discriminator and gets classified as either “Real” or “Fake” based on the ability of the discriminator to tell one from the other.

- The generator loss is then calculated from the discriminator’s classification – it gets rewarded if it successfully fools the discriminator, and gets penalized otherwise.

non-Saturating Generator Loss

- A subtle change in the standard loss function can be where the generator maximizes the log of the discriminator probabilities
- $\log(D(G(z)))$.

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] - \mathbb{E}_{z \sim p_z(z)} [\log(-D(G(z)))].$$

- This change is inspired by framing the problem from a different perspective, where
 - the generator seeks to maximize the probability of images being real,
 - instead of minimizing the probability of an image being fake.

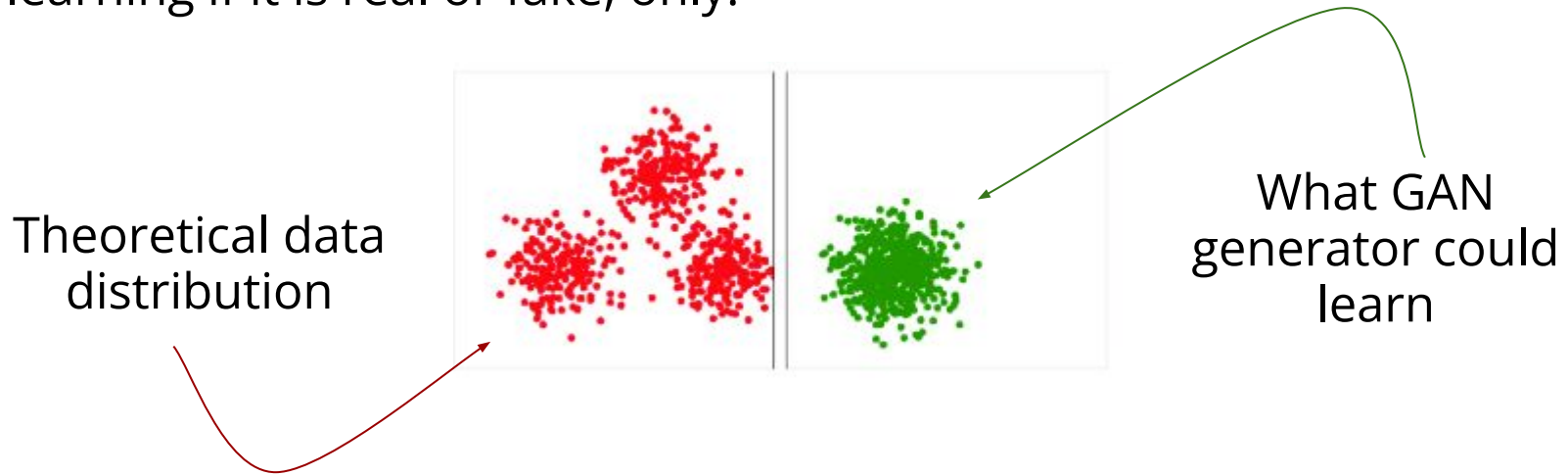
Mode Collapse

- Usually, we would want our GAN to produce a range of outputs. We would expect, for example, another face for every random input to the face generator that we design.
- But the loss function and architecture of the GAN Loss functions has NO inherent component to push for this diversity.
- Hence GANs may end up in a trivial scenario where, the network learns to model a particular distribution of data, which gives us a monotonous output which is illustrated below.



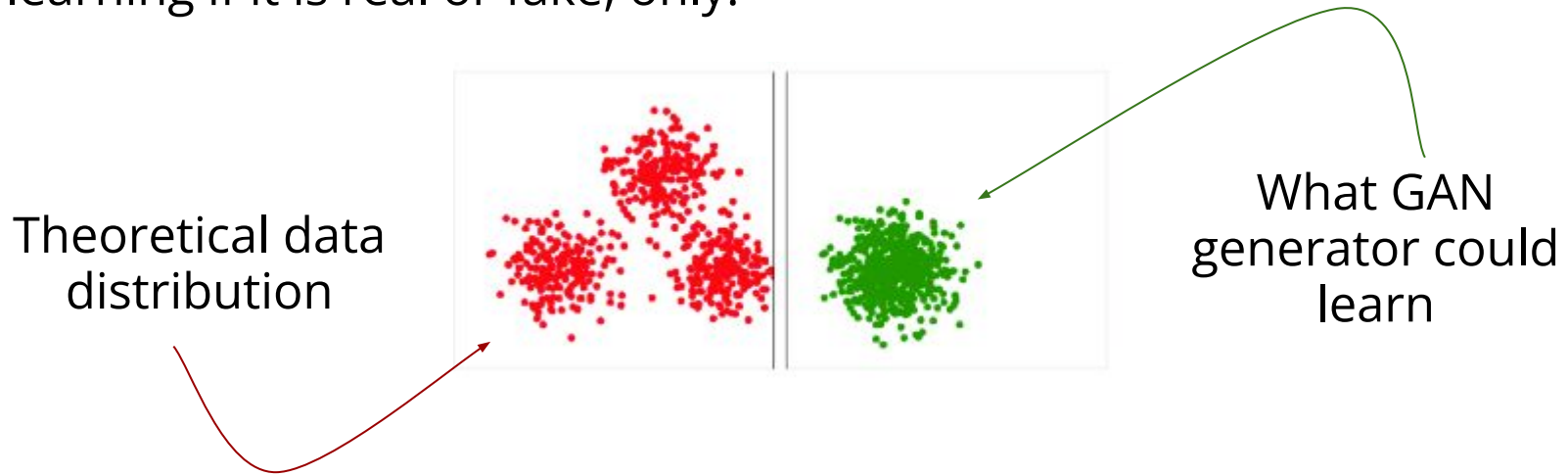
Mode Collapse

- Fundamentally, mode collapse is about not learning the entire distribution.
- In GANs there is no mechanism to learn the distribution. It is about learning if it is real or fake, only.



Mode Collapse

- Fundamentally, mode collapse is about not learning the entire distribution.
- In GANs there is no mechanism to learn the distribution. It is about learning if it is real or fake, only.



GAN Failure modes

- Vanishing gradients: The discriminator being forced to output $[0,1]$ probabilities (i.e. logits) causes old-fashioned problems.
- Convergence : The utopian situation where both networks stabilize and produce a consistent result (simultaneously) is hard to achieve in most cases.
 - Perfect Discriminator: No gradients reach the Generator.
 - Poor Discriminator: Discriminator can not learn the data realism well (e.g. many eyed dogs problem)
- Mode Collapse: Generator learns a part of the data distribution only.

How to solve GAN Failure modes

- This has been the main GAN research in the recent years.
- cGANs, by nature propose some level of improvement, because they push the discriminator to be a better model.
- But by changing the loss function, better results can be achieved.
- The most well-known of these approaches is the Wasserstein loss.
- W-Loss tackles both the problems of Mode Collapse and Vanishing Gradient.

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

Wasserstein Loss

- Also known as Earth Mover's Distance or Wasserstein Distance.
- Measures the distance between two probability distributions.

W-Loss approximates the Earth Mover's Distance

$$\min_g \max_c \mathbb{E}(c(x)) - \mathbb{E}(c(g(z)))$$


Generator

Minimize the distance

Critic

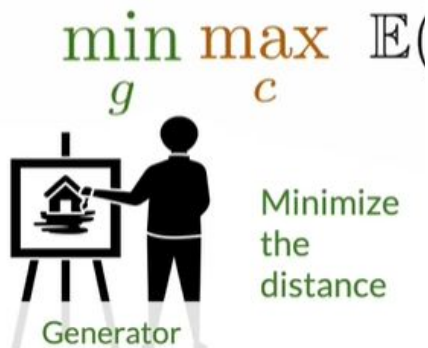
Maximize the distance



Wasserstein Loss

- Also known as Earth Mover's Distance
- Measures the distance between

W-Loss approximates the Earth Mover's Distance



$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

When you are researcher at Facebook AI (now META) you don't care if you have or have not an abstract in your papers.

Wasserstein GAN

Martin Arjovsky¹, Soumith Chintala², and Léon Bottou^{1,2}

¹Courant Institute of Mathematical Sciences

²Facebook AI Research

1 Introduction

The problem this paper is concerned with is that of unsupervised learning. Mainly, what does it mean to learn a probability distribution? The classical answer to this is to learn a probability density. This is often done by defining a parametric family of densities $(P_\theta)_{\theta \in \mathbb{R}^d}$ and finding the one that maximized the likelihood on our data: if we have real data examples $\{x^{(i)}\}_{i=1}^m$, we would solve the problem

$$\max_{\theta \in \mathbb{R}^d} \frac{1}{m} \sum_{i=1}^m \log P_\theta(x^{(i)})$$

for more on GAN failure modes,
[check this one](#) and [this one](#)

Wasserstein Loss

- What is $c(x)$?
- We don't use the original Wasserstein formula.

W-Loss approximates the **Earth Mover's Distance**

$$\min_g \max_c \mathbb{E}(c(x)) - \mathbb{E}(c(g(z)))$$



Generator

Minimize
the
distance



Critic

Maximize
the
distance

Wasserstein Loss

- What is $c(x)$?
- We don't use the original Wasserstein formula.
- The original Wasserstein distance formula involves an optimization problem that might be computationally expensive. In the context of GANs, the Wasserstein distance is approximated by using a relaxed version, which leads to the Wasserstein loss.
- We just don't bound the $D(x)$ input between 0-1 with a sigmoid

Wasserstein Loss

W-Loss vs BCE Loss

BCE Loss	W-Loss
Discriminator outputs between 0 and 1	Critic outputs any number
$-\left[\mathbb{E}(\log(d(x))) + \mathbb{E}(1 - \log(d(g(z))))\right]$	$\mathbb{E}(c(x)) - \mathbb{E}(c(g(z)))$

- We just don't bound the $D(x)$ input between 0-1 with a sigmoid
- Instead linear activation layer (not ReLU, just linear, like nothing)
 - Output between $[-\infty, \infty]$

Wasserstein Loss

- Why is it called in original Arjovsky's paper? It has nothing to do with the original Wasserstein distance!
- How does it help with the mode collapse issue?
- What is the the new loss function exactly?

Let's make a summary of the idea by answering the above questions.

Wasserstein Loss

- In Arjovsky's paper, the concept is formally introduced as the Wasserstein distance or Earth Mover's distance (EMD).
- The paper describes a modification in GAN training by approximating this distance, which measures how much "mass" has to be transported to change one probability distribution into another.

Wasserstein Loss

- The Wasserstein distance is a real metric for probability distributions and does not have the same properties as the loss functions in traditional GANs.
- The main difference is that it does not require overlapping supports of real and generated data distributions to compute gradients, so it remains informative even in situations where traditional GANs struggle.
- In practice, Arjovsky's method does not compute the exact Wasserstein distance but rather an approximation, which improves training and convergence.

Wasserstein Loss

- Wasserstein GANs (WGANs) help with mode collapse because the Wasserstein loss penalizes distances between the real and generated distributions in a smoother way.
- Unlike in standard GANs where the generator might get “stuck” producing a limited set of samples, the Wasserstein distance gives more meaningful gradients, helping the generator to explore more of the data distribution and thus cover different modes more effectively.

W-Loss Implementational Details

- The paper has many intricate implementation details
 - Constrain critic model weights to a limited range after each mini batch update (e.g. $[-0.01, 0.01]$).
 - Update the critic model more times than the generator each iteration (e.g. 5).
 - Use the RMSProp version of gradient descent with small learning rate and no momentum (e.g. 0.00005).

What will we do next week?

- GANs continued...